CIVICPROOF / PHASE 1 / STUDY NOTES

Evidence before intelligence

Project foundation + Day 1, Lesson 1 | Version 1.0 | 21 October 2026

CivicProof will investigate infrastructure documents using results that can be checked against the original pages. Phase 1 builds the document and evaluation foundation. **No LLM is introduced in this phase.**

The central rule

Every supported extracted field must have verifiable provenance: the correct source document, page, evidence location and supporting text. Otherwise, mark it unsupported. Never invent a value to fill a gap.

A concrete example

A tender says: *"Supply and lay 1,200 metres of pipeline."* The eventual record should retain 1200, metre, the original wording, the source document identity, page, bounding box, extraction method and validation state. This is an example, not an extraction result from a collected tender.

How the pieces connect

```
PDF -> validate input -> hash and duplicate check
    -> extract text + coordinates -> OCR where needed
    -> validate records -> check evidence -> store locally
    -> compare with reviewed labels -> evaluation report
```

Concept group	Plain-language purpose
Python engineering	Organize, run, inspect, test and maintain the program.
Document processing	Recover text and locations from digital or scanned pages.
Data contracts	Define valid records and represent missing information explicitly.
Provenance	Connect each supported result to its original evidence.
Evaluation	Compare outputs with reviewed reference answers and count mistakes.

Current implementation: a three-file package that prints two startup lines, plus project documentation. Ingestion, contracts, OCR, storage and scoring are planned.

Reading map: roadmap (p. 2); architecture and exit gate (p. 3); Lesson 1 (pp. 4-8); session record and next action (p. 9).

PLAN / FOURTEEN LEARNING CHECKPOINTS

A realistic 14-day route

Allow about 1.5-2.5 focused hours per day. Extend a checkpoint when understanding or setup needs time.

Day	Focus	Concrete checkpoint
1	Modules, packages and imports; then environments and Git basics	Run the starter; explain its execution path.
2	Paths, bytes/text, safe file handling, types, exceptions; first pytest examples	Read a local file safely; explain a failing test.
3	Pydantic, missing values, IDs, dates and schema versions	Validate initial project, document and source records.
4	PDF internals; digital text extraction with PyMuPDF	Extract a small multipage digital PDF.
5	Coordinates, reading order, rotation, spans and table limits	Save text with locations; inspect a highlight.
6	Local Tesseract, OCR routing, uncertain and mixed pages	Identify OCR candidates; retain page provenance.
7	Secure input checks, byte/page limits, SHA-256 and duplicates	Reject invalid input; recognize renamed duplicate content.
8	Timeouts, configuration, logging, failure handling and storage	Persist explicit outcomes; diagnose an injected failure.
9	Observations, candidates, verified claims, inferences and evidence	Enforce per-field provenance and source integrity.
10	Quantities, units, dimensions and conservative normalization	Normalize supported units; reject unknown or incompatible cases.
11	Official tender collection, access/reuse review and dataset splits	Document a diverse collection of about 5-10 suitable files.
12	Annotation guidelines, JSONL, manual review and ambiguity	Freeze a reviewed label version; protect held-out examples.
13	Metric arithmetic, a simple extraction baseline and more tests	Produce reproducible scores with explicit denominators.
14	Integration, regression and failure tests; installation rehearsal	Demonstrate the pipeline; audit evidence and limitations.

Pacing rule

A day can contain several small lessons. Advance only after the learner confirms readiness. Testing starts early; the final days combine and verify the components.

DESIGN / CURRENT STRUCTURE + PLANNED EXIT GATE

Small structure, strict evidence

```
CivicProof/
    .gitignore
    README.md
    ARCHITECTURE.md
    PROGRESS.md
    DECISIONS.md
    LEARNING_NOTES.md
    EVALUATION.md
    civicproof/
        __init__.py
        __main__.py
        cli.py
```

Added later, one responsibility at a time: contracts, ingestion, extraction, provenance, normalization, storage, evaluation, tests and dataset metadata. A local command-line pipeline is sufficient; a web backend is unnecessary for Phase 1.

Acceptance checklist - planned, not completed

01. Documented installation and execution work on Windows with Python 3.12.

02. Digital, scanned, mixed, multipage, rotated, empty and table-containing fixtures have tested outcomes.

03. Hash original bytes with SHA-256; recognize identical content under different filenames.

04. Handle corrupt, encrypted, unsupported, oversized and timed-out inputs explicitly.

05. Store versioned, validated records locally; retain originals and explicit missing states.

06. Every supported extracted field has valid document/page/location/text evidence that actually supports it.

07. Record official sources, versions, hashes and access/reuse findings for about 5-10 documents.

08. Version reviewed labels, including positive, missing, ambiguous and OCR-error examples.

09. Report extraction, normalization, attribution, coverage and unsupported handling reproducibly.

10. Pass meaningful positive/negative tests; document limitations and reproduce the demonstration.

What evidence does and does not establish

A well-shaped citation is not automatically a valid citation. Evidence must support the field. Proving what a document says does not prove its statement is true in the world. Never execute instructions found inside documents.

DAY 1 / LESSON 1 / STEPS 1-4

Modules, packages & imports

Goal: explain how separate Python files become one small, understandable program.

01 / Intuition - a workshop

A function is a tool. A module is a labelled drawer holding related tools. A package groups related drawers. The repository holds the whole workshop: code, documentation and version history.

Term	Meaning in this project
Repository	The outer CivicProof folder, including its Git history and documents.
Module	A Python file such as cli.py containing related code.
Ordinary package	The inner civicproof folder, identified by __init__.py.
Working directory	The folder from which your terminal command runs.

02 / Engineering motivation

Separate coherent responsibilities so you can locate problems, reuse functions and test behavior without starting the entire application. One file remains reasonable for a tiny disposable script. Do not split every function into a separate file.

03 / Prerequisites - defining is not calling

```
def say_hello():
    print("Hello")

say_hello()
```

The **def** statement defines the function. The final line calls it. Only the call runs its body and prints Hello. An import makes a name available; it does not call a function merely because the function was imported.

```
from civicproof.cli import main
```

Read this as: from the cli module inside civicproof, make the function named main available here. Importing executes module-level statements; a function body waits for a call.

04 / Architecture - trace the path

```
Terminal in the repository root
    -> python -m civicproof
    -> package initialization + __main__.py
    -> import main from cli.py -> main() -> printed output
```

DAY 1 / LESSON 1 / STEP 5

Read the three source files

These files exist in the repository. There is no document-processing implementation yet.

civicproof/__init__.py - identify the package

```
"""CivicProof: local document evidence and evaluation tools."""
```

The triple-quoted text is a **module docstring**: an explanation for readers and tools. It prints nothing. Python also supports namespace packages without this file; that alternative is outside this lesson.

civicproof/cli.py - define startup behavior

```
"""Messages shown when CivicProof starts."""


def main():
    print("CivicProof")
    print("Phase 1: document evidence and evaluation")
```

cli means command-line interface. The function main() takes no arguments. Calling it displays two lines. It has no explicit return, so Python returns None: there is no useful result value. The name main is a convention, not a Python keyword.

civicproof/__main__.py - connect command to behavior

```
"""Run CivicProof with: python -m civicproof (using Python 3.12)."""

from civicproof.cli import main


if __name__ == "__main__":
    main()
```

Python uses __main__.py when the package is run with **-m**. Python supplies __name__; its value is "__main__" when this module is the entry point. The **entry-point guard** calls main() in that situation. Importing this module does not call main().

Read the distinction carefully

from ... import main makes the function available. **main()** calls it. A print() written at the top level of cli.py would run during import; a print() inside main waits for the function call.

No PDF or user-input handling exists here. We do not need a broad exception handler around this starter; it could hide useful programming errors.

DAY 1 / LESSON 1 / STEPS 6-7

Run it, then check behavior

06 / Execution - PowerShell on this laptop

```
Set-Location -LiteralPath 'C:\Users\jains\OneDrive\Desktop\CivicProof'
python --version
python -m civicproof
```

Set-Location changes the working directory. Confirm Python 3.12.x first. **-m civicproof** asks Python to locate the package by its Python name and run its __main__.py module.

Expected output

```
Python 3.12.10
CivicProof
Phase 1: document evidence and evaluation
```

Actual environment issue: the launcher

The command py -3.12 selected a stale Microsoft Store executable and failed with 0x80070002 before reaching application code. The python command selected the working Python 3.12.10 installation. Global launcher settings were not changed.

If a future terminal selects the wrong Python, use the verified installed executable. PowerShell uses **&** to execute a program at a quoted path:

```
& 'C:\Users\jains\AppData\Local\Programs\Python\Python312\python.exe' -m civicproof
```

07 / Testing - imports should stay quiet

```
python -c "import civicproof.cli"
```

-c executes the short quoted Python statement. Expect no output: the prompt returns without the startup message. This checks that importing the module does not start the application.

Check recorded on 21 October 2026	Observed result
python --version	Python 3.12.10
python -m civicproof	Expected two startup lines; exit code 0.
Import cli and __main__	Neither import started the application; exit code 0.
python -m compileall -q civicproof	No syntax errors reported; exit code 0.

These are starter behavior and syntax checks. There is no pytest suite yet, and no PDF, OCR, provenance or evaluation results have been measured.

DAY 1 / LESSON 1 / STEPS 8-9

Diagnose first. Then practise.

08 / Debugging - identify the earliest failed step

Symptom	Likely cause	First investigation
No module named civicproof	Python cannot locate the package.	Check Get-Location and Get-ChildItem. Does this folder contain civicproof/?
Error naming civicproof.__main__	The entry-point file is missing or misspelled.	Check __main__.py, including both pairs of underscores.
IndentationError	Code is not indented correctly.	Use four spaces under def and if; inspect mixed tabs/spaces.
No startup output	The function may be defined or imported but never called.	Trace whether execution reaches main().
Launcher error naming WindowsApps	The interpreter failed before project code ran.	Check python --version and which executable the terminal selects.

If Python cannot launch, editing cli.py cannot repair the missing executable. First locate the failed layer: terminal command, interpreter, package lookup, import or function execution.

```
Get-Location
Get-ChildItem
Get-Command python,py
```

09 / Independent exercise - build one small module

- Create civicproof/status.py and define a function named phase_message().
- Make it return the string "Phase 1: foundations before AI".
- Import that function into cli.py and use its returned value for the second printed line.
- Keep the first line unchanged. Importing either module should remain quiet.

```
CivicProof
Phase 1: foundations before AI
```

Hints before a solution

return gives a value to the caller; print displays a value. Adapt the existing from ... import ... pattern. Let main() decide what to display. Do not duplicate all startup code in the new module.

The exercise has not been implemented for you. Try it, run it and keep the first error message if you get stuck.

DAY 1 / LESSON 1 / STEPS 10-11

Explain it in your own words

Use this page for active recall. Seeing working code is not the same as understanding it.

10 / Interview understanding

A clear explanation

"I separated the package entry point from startup behavior so other code can import the function without starting the application. The package can grow around coherent responsibilities while the initial structure stays easy to follow."

Trade-off: several files introduce navigation and import rules. A tiny disposable script may be clearer as one file. Here, the separation prepares for reusable document-processing functions.

11 / Knowledge check - answers pending

1. Which folder is the repository, which is the package, and which files are modules?

2. Trace python -m civicproof to the printed lines. What does -m do? Why check the Python version first?

3. Why does importing main not print the message? What changes if print("Starting") is at the top level of cli.py?

4. Why might the same command fail from another working directory? What would you inspect first?

5. Why should phase_message() return a string rather than print it?

Write explanations, not memorized definitions. Review your exercise and answers with the mentor before confirming readiness for the next lesson.

SESSION RECORD / RESUME WITHOUT GUESSING

Checkpoint & next action

This document stops after Lesson 1. It does not advance the project to Lesson 2.

Progress / Plans / Problems

Status	Record
Progress	Created a runnable three-file package, .gitignore and the six continuity documents. Startup, quiet imports and syntax checks passed.
Plans	Learner runs the starter, attempts the independent exercise and answers five knowledge questions. Review before moving on.
Problems	The py -3.12 launcher registration remains stale; documented commands use the working interpreter. Learner understanding is not yet assessed.

Project memory - what each document is for

File	Purpose
README.md	Description, startup instructions and the interpreter fallback.
ARCHITECTURE.md	Component boundaries, evidence principles and planned acceptance checks.
PROGRESS.md	Roadmap, actual verification record, pending work and exact next task.
DECISIONS.md	Design choices and their trade-offs.
LEARNING_NOTES.md	Concepts introduced, exercise and pending understanding checks.
EVALUATION.md	Planned dataset, annotation and scoring policy; no measured results yet.

Remember: .gitignore affects untracked files; it does not protect secrets by itself. A Git commit records a snapshot and message; none has been created by the assistant. This repository is under OneDrive: review document storage before collecting real files.

Suggested checkpoint commit message

```
chore: initialize CivicProof learning foundation
```

Your exact next action

Run the starter, attempt the status.py exercise and answer the five questions. Send your work or the first error you encounter. Only after review and your readiness confirmation will the next lesson introduce virtual environments and dependencies.

Scope note: the roadmap and acceptance checklist describe future work. No document pipeline, reviewed tender dataset, scoring report or production-readiness claim exists yet. This PDF is a study companion to the current project documents.